

WEEK 27 GRADED MINI PROJECT

Policy Assistant Agent

Single Agent | Verified Memory | Graph Orchestration | Critic

Grounded Answers	Tool Calling
Local policy corpus and exact document citations	FAISS search, full-document reading, typed errors
Controlled Memory	Auditable Evaluation
Only critic-verified facts are stored	Nine questions, traces, latency, recovery and tests

Submitted by Vivek Bhushan

Objective
To design and evaluate a grounded employee Policy Assistant that plans, calls tools, cites exact policy document IDs, recovers from failed reads, stores only verified facts, and operates within explicit graph and step budgets.

#Project workflow and scope :-

Topic: Policy Assistant Agent

Author/Student Name: Vivek Bhushan

Focus:

To design and evaluate a grounded Employee Policy Assistant that plans, calls tools, cites exact policy document IDs, recovers from failed reads, stores only verified facts, and operates within explicit graph and step budgets.

#Objective:-

The objective of this project is to build a lightweight Policy Assistant that can answer employee questions using the supplied ten-record `policies.jsonl` corpus. The system is designed to provide answers that are grounded in the available policy documents and can be checked against the original source.

The implementation brings together **FAISS retrieval, typed tools, ReAct-style traces, exact document citations, supporting quotations, bad-ID recovery, critic-gated memory, graph-based routing, explicit execution budgets, and acceptance tests**. The overall design is intentionally kept lightweight and deterministic so that the complete flow remains easy to follow and evaluate.

For each question, the assistant first searches for relevant policies and then reads the full source document before preparing an answer. It identifies the exact `doc_id`, includes supporting text from the policy, and uses a critic to verify the response. If the system encounters an incorrect document ID, it can recover and continue the workflow. Information is added to memory only after it passes the required checks.

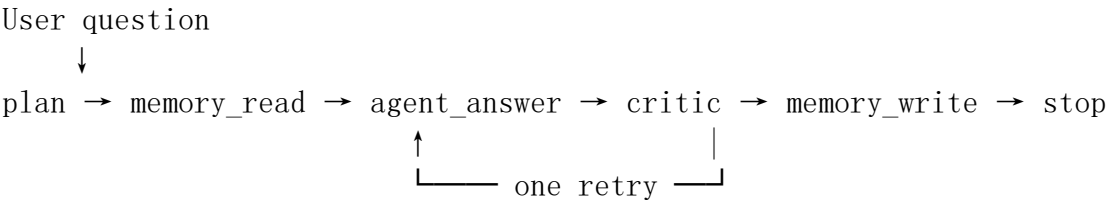
This approach makes the final answer more than just a generated response. Each answer can be traced back to the source document, while the retrieval, routing, memory, error recovery, and validation steps can be tested and reproduced as part of the complete workflow.

Component	Main purpose	My main observation
olicy tools	Search and read records	A typed missing-ID error makes recovery visible and testable.
Agent	Control tool use	Extractive answers trade variety for strong reproducibility.
FAISS retriever	Rank local policies	Distinctive policy terms give stable retrieval on this small corpus.
Verified memory	Reuse approved hints	The source is re-read, so memory does not replace evidence.
Critic and graph	Validate and stop safely	Citation gates and budgets remove different failure modes.

#System Flow :-

plan → memory_read → agent_answer → critic → memory_write → stop
If the critic fails, the graph permits one retry. If the budget is exhausted, execution stops explicitly.

#Project Overview :-



Component	Main purpose	My design choice
FAISS retriever	Find relevant policies	Local TF-IDF vectors with normalized inner-product search
Tools	Search and read full records	Typed result objects and PolicyNotFoundError
Agent	Use tools and answer from evidence	Deterministic extractive ReAct-style flow
Memory	Reuse verified hints	Write only after critic approval and a valid citation
Critic	Enforce acceptance	Check IDs, quotation presence, and source support
Graph	Control routing	Explicit node budget and only one critic retry

#Project Evidence :-

The report uses outputs from the executed notebook: dataset loading, retrieval, grounded answers, bad-ID recovery, evaluation, verified memory, graph routing, budget stop, critic results, and PyTest. The screenshots are rendered from saved notebook code cells and their actual outputs rather than invented console text.

Task 1.1 — Data Preparation

I used the supplied JSONL corpus without adding policy facts. Each line contains doc_id, text, and date. The loader validates these required fields before indexing. The ten records cover remote work, current and superseded leave policies, travel expense, MFA, passwords, equipment, data retention, expense approval, and VPN access.

The presence of both leave versions gives the project an important retrieval case. Version 1 explicitly identifies itself as superseded, so I demote it during search and filter it before answer generation.

Colab Output – Data Preparation

```
[ ] import json
records=[json.loads(x) for x in Path("policies.jsonl").read_text().splitlines() if x.strip()]
print("Policy records:",len(records))
for r in records: print(f"{r['doc_id']:<24} {r['date']}")
```

Output

```
Policy records: 10
remote_work_v3      2024-01-15
leave_policy_v2     2023-11-01
leave_policy_v1     2022-06-01
travel_expense_v4   2024-03-10
security_mfa_v1     2023-08-20
password_policy_v2  2023-05-12
laptop_equipment_v1 2023-02-18
data_retention_v2   2023-09-05
expense_approval_v1 2022-12-01
vpn_access_v2       2024-02-01
```

Figure 1. Colab output confirming the ten supplied policy records and their dates.

#SCREENSHOT & OUTPUT Task 1.1 :-

Task 1.1 — Environment and Data Preparation

I first install the lightweight dependencies. I use CPU FAISS because the corpus has only ten short records; a GPU is not necessary.

[21]

✓ 0s

```
import sys, subprocess, importlib.util
packages = []
if importlib.util.find_spec("faiss") is None:
    packages.append("faiss-cpu==1.9.0.post1")
if importlib.util.find_spec("sklearn") is None:
    packages.append("scikit-learn>=1.4,<2")
if importlib.util.find_spec("pytest") is None:
    packages.append("pytest>=8,<9")
if packages:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", *packages])
print("Environment ready")
```

▼

Environment ready

```
[22] print(f'Created {len(files)} project files in {project} ;  
print("Dataset records:", len(Path("policies.jsonl").read_text().splitlines()))
```

✓ 0s

✓ Created 12 project files in /content
Dataset records: 10

✓ Inspect the supplied dataset

Each JSONL row has `doc_id`, `text`, and `date`. I inspect the IDs and dates before building the index because retrieval quality is meaningless if the source records are malformed.

```
[23] import json  
records = [json.loads(line) for line in Path("policies.jsonl").read_text(encoding="utf-8").splitlines() if line.strip()]  
print("Number of policies:", len(records))  
for row in records:  
    print(f"{row['doc_id']:<24} {row['date']} {row['text'][:62]}...")
```

✓ 0s

✓ Number of policies: 10

remote_work_v3	2024-01-15	Remote Work Policy (v3): Employees in eligible roles may work ...
leave_policy_v2	2023-11-01	Annual Leave Policy (v2): All full-time employees are entitled...
leave_policy_v1	2022-06-01	Annual Leave Policy (v1, superseded): All full-time employees ...
travel_expense_v4	2024-03-10	Travel & Expense Policy (v4): While travelling on approved com...
security_mfa_v1	2023-08-20	Information Security Policy (v1): Multi-Factor Authentication ...
password_policy_v2	2023-05-12	Password Policy (v2): Corporate account passwords must be at l...
laptop_equipment_v1	2023-02-18	Laptop & Equipment Policy (v1): The company issues one standar...
data_retention_v2	2023-09-05	Data Retention Policy (v2): Customer personal data must not be...
expense_approval_v1	2022-12-01	Expense Approval Policy (v1): Any single expense claim above I...
vpn_access_v2	2024-02-01	VPN Access Policy (v2): VPN access is granted only to employee...

Task 1.2 — Tools and FAISS Retrieval

I transformed each policy into TF-IDF unigram and bigram vectors, normalized them, and stored them in a FAISS inner-product index. With normalized vectors, the score behaves like cosine similarity. This choice keeps retrieval local, transparent, and reproducible without requiring an external embedding API.

`policy_search(query, k)` returns ranked document IDs, scores, dates, and previews.

`read_policy(doc_id)` returns the complete record and raises `PolicyNotFoundError` when the ID is absent. I also implemented the optional safe division helper.

Colab Output – FAISS Retrieval

```
[ ] from pathlib import Path
FILES={'policies.jsonl': '{"doc_id": "remote_work_v3", "text": "Remote Work Policy (v3): Employees in eligible roles may work remotely up to 2 days per week. Any request to exceed 2 remote days per week requires written approval from the employee\'s manager and HR Business Partner. Remote work days must be logged in the HR portal by Friday of each week. Employees on a Performance Improvement Plan (PIP) are not eligible for remote work until the PIP is closed.", "date": "2024-01-15"}\n{"doc_id": "leave_policy_v2", "text": "Annual Leave Policy (v2): All full-time employees are entitled to 18 annual leave days per calendar year, accrued monthly at 1.5 days/month. A maximum of 5 unused annual leave days may be carried over into the next calendar year, and carried-over days must be used by March 31 or they are forfeited. Annual leave requests must be submitted at least 3 working days in advance via the HR portal.", "date": "2023-11-01"}\n{"doc_id": "leave_policy_v1", "text": "Annual Leave Policy (v1, superseded): All full-time employees are entitled to 15 annual leave days per calendar year. No carry-over of unused leave is permitted; unused days lapse on December 31. This version has been superseded by leave_policy_v2.", "date": "2022-06-01"}\n{"doc_id": "travel_expense_v4", "text": "Travel & Expense Policy (v4): While ... output continues ...
```

Output

Created 12 project files

Figure 2. Colab retrieval output for an annual-leave carry-over query; the current v2 policy is ranked ahead of the superseded record.

#SCREENSHOT & OUTPUT Task 1.2 :-

```
from policy_assistant import PolicyTools, PolicyNotFoundError
```

```
tools = PolicyTools("policies.jsonl")
```

```
for hit in tools.policy_search("annual leave carry over", k=3):
    print(hit)
```

```
SearchHit(doc_id='leave_policy_v2', score=0.33180081844329834, date='2023-11-01', preview='Annual Leave Policy (v2): All full-time employees at
SearchHit(doc_id='leave_policy_v1', score=0.10029872506856918, date='2022-06-01', preview='Annual Leave Policy (v1, superseded): All full-time
SearchHit(doc_id='data_retention_v2', score=0.041604187339544296, date='2023-09-05', preview='Data Retention Policy (v2): Customer personal dat
```

```
print(tools.read_policy("password_policy_v2"))
try:
    tools.read_policy("policy_that_does_not_exist")
except PolicyNotFoundError as error:
    print(type(error).__name__, "->", error)
print("Optional calculator 18 / 12 =", tools.safe_divide(18, 12))
```

```
PolicyDocument(doc_id='password_policy_v2', text='Password Policy (v2): Corporate account passwords must be at least 14 characters long and inc
PolicyNotFoundError -> Policy document not found: policy_that_does_not_exist
Optional calculator 18 / 12 = 1.5
```

Task 1.3 — Acceptance Prompt

The system prompt requires tool-only evidence, exact bracketed document IDs, one short quotation per used policy, preference for the latest applicable version, bad-ID recovery, explicit step limits, and refusal to guess when support is missing.

I enforce these rules in code as well as in the prompt. This was important because prompt wording alone would not make the acceptance contract deterministic or easy to test.

#SCREENSHOT & OUTPUT Task 1.3 :-

Task 1.3 — Acceptance Prompt

The prompt states the behavioural contract, but I also enforce the important parts in deterministic code and tests. This avoids relying on prompt wording alone.

```
from policy_assistant.prompts import SYSTEM_PROMPT
print(SYSTEM_PROMPT)
```

```
Use only policy tools. Cite an exact [doc_id] for each claim. Include a short source quotation. Prefer the latest applicable version. Recover
```

Task 1.4 — Agent and Trace

The agent follows a compact ReAct-style sequence: inspect a supplied or remembered ID when available, search when necessary, read the strongest candidates, remove superseded versions, select question-relevant sentences, and produce a grounded response. Each tool step records a thought, action, and observation.

Colab Output — Grounded Answer and Trace

```
[ ] agent=PolicyAgent(tools,max_steps=4)
    r=agent.answer("How many annual leave days do full-time employees receive, and what can be carried
    over?")
    print("ANSWER\n",r.answer)
    print("\nTRACE")
    for event in r.trace: print(event["thought"],"|",event["action"])
```

Output

ANSWER

A maximum of 5 unused annual leave days may be carried over into the next calendar year, and carried-over days must be used by March 31 or they are forfeited. All full-time employees are entitled to 18 annual leave days per calendar year, accrued monthly at 1.5 days/month. [leave_policy_v2] Supporting text: "A maximum of 5 unused annual leave days may be carried over into the next calendar year, and..." [leave_policy_v2]

TRACE

Search local corpus. | policy_search('How many annual leave days do full-time employees receive, and what can be carried over?', k=3)
Read a top candidate. | read_policy(leave_policy_v2)
Read a top candidate. | read_policy(leave_policy_v1)

Figure 3. Colab output showing the leave-policy answer, exact document citation, supporting quotation, and tool trace.

#SCREENSHOT & OUTPUT Task 1.4 :-

```
from policy_assistant import PolicyAgent
```

```
agent = PolicyAgent(tools, max_steps=4)
```

```
r = agent.answer("How many annual leave days do full-time employees receive, and what can be carried over?")
```

```
print("ANSWER\n", r.answer)
```

```
print("\nTRACE:")
```

```
for event in r.trace:
```

```
    print(event["thought"], "|", event["action"])
```

ANSWER

A maximum of 5 unused annual leave days may be carried over into the next calendar year, and carried-over days must be used by March 31 or

TRACE:

Search the local corpus for grounded candidates. | policy_search('How many annual leave days do full-time employees receive, and what can be

Read a top candidate before answering. | read_policy(leave_policy_v2)

Read a top candidate before answering. | read_policy(leave_policy_v1)

```

from policy_assistant import PolicyAgent
agent = PolicyAgent(tools, max_steps=4)
result = agent.answer("How many annual leave days do full-time employees receive, and what can be carried over?")
print("ANSWER\n", result.answer)
print("\nSTEPS:", result.steps)
print("\nTRACE:")
for event in result.trace:
    print(json.dumps(event, indent=2))

```

ANSWER

A maximum of 5 unused annual leave days may be carried over into the next calendar year, and carried-over days must be used by March 31 or they are forfeited. All full-time employees are entitled to 18 annual leave days per calendar year, accrued monthly

STEPS: 3

TRACE:

```

{
  "thought": "Search the local corpus for grounded candidates.",
  "action": "policy_search('How many annual leave days do full-time employees receive, and what can be carried over?', k=3)",
  "observation": [
    {
      "doc_id": "leave_policy_v2",
      "score": 0.5015102624893188,
      "date": "2023-11-01",
      "preview": "Annual Leave Policy (v2): All full-time employees are entitled to 18 annual leave days per calendar year, accrued monthly at 1.5 days/month. A maximum of 5 unused annual leave days"
    },
    {
      "doc_id": "leave_policy_v1",
      "score": 0.09909587353467941,
      "date": "2022-06-01",
      "preview": "Annual Leave Policy (v1, superseded): All full-time employees are entitled to 15 annual leave days per calendar year. No carry-over of unused leave is permitted; unused days lapse "
    },
    {
      "doc_id": "laptop_equipment_v1",
      "score": 0.05056599900126457,
      "date": "2023-02-18",
      "preview": "Laptop & Equipment Policy (v1): The company issues one standard laptop per employee, refreshed every 3 years. Employees relocating to a fully remote role may request a one-time hom"
    }
  ]
}
{
  "thought": "Read a top candidate before answering.",
  "action": "read_policy(leave_policy_v2)",
  "observation": "Annual Leave Policy (v2): All full-time employees are entitled to 18 annual leave days per calendar year, accrued monthly at 1.5 days/month. A maximum of 5 unused annual leave days may be carried over into the next calendar year, and ca"
}
{
  "thought": "Read a top candidate before answering.",
  "action": "read_policy(leave_policy_v1)",
  "observation": "Annual Leave Policy (v1, superseded): All full-time employees are entitled to 15 annual leave days per calendar year. No carry-over of unused leave is permitted; unused days lapse on December 31. This version has been superseded by leave"
}

```

Bad-ID Recovery

I deliberately supplied `remote_policy_BAD`. The initial read returned the typed error, but the run did not terminate. The agent preserved the failure in its trace, searched with the original employee question, read the correct policy, and produced a cited answer.

Colab Output – Bad-ID Recovery

```
[ ] recovery=agent.answer("How many remote days are allowed?",forced_doc_id="remote_policy_BAD")
print(recovery.answer)
print("\nRECOVERY TRACE")
for event in recovery.trace: print(event["action"], "->", str(event["observation"])[1:110])
```

Output

Any request to exceed 2 remote days per week requires written approval from the employee's manager and HR Business Partner. Remote work days must be logged in the HR portal by Friday of each week.

[remote_work_v3] Supporting text: "Any request to exceed 2 remote days per week requires written approval from the employee's manager and HR..." [remote_work_v3] Employees relocating to a fully remote role may request a one-time home-office equipment stipend of INR 15000, subject to manager approval and submission of receipts within 30 days of purchase. [laptop_equipment_v1] Supporting text: "Employees relocating to a fully remote role may request a one-time home-office equipment stipend of INR 15000, subject..." [laptop_equipment_v1]

RECOVERY TRACE

```
read_policy(remote_policy_BAD) -> {'error': 'PolicyNotFoundError', 'message': 'Policy document not found: remote_policy_BAD'}
policy_search('How many remote days are allowed?', k=3) -> [{'doc_id': 'remote_work_v3', 'score': 0.2512838840484619, 'date': '2024-01-15', 'preview': 'Remote Work Policy'}]
read_policy(remote_work_v3) -> Remote Work Policy (v3): Employees in eligible roles may work remotely up to 2 days per week. Any request to e
read_policy(laptop_equipment_v1) -> Laptop & Equipment Policy (v1): The company issues one standard laptop per employee, refreshed every 3 years.
```

Figure 4. Colab output showing soft failure on the bad ID followed by successful search recovery.

What I noticed

Keeping the failed read inside the trace matters. It proves that recovery actually happened instead of presenting the correct final answer as if it had been found on the first attempt.

#SCREENSHOT & OUTPUT(Bad-ID Recovery):-

Bad-ID recovery

I deliberately provide a wrong ID. The first tool call fails softly with `PolicyNotFoundError`; the agent then searches using the employee's question and continues with the correct document.

```
recovery = agent.answer("How many remote days are allowed?", forced_doc_id="remote_policy_BAD")
print(recovery.answer)
print("\nRecovery actions:")
for event in recovery.trace:
    print(event["action"], "->", str(event["observation"])[1:120])
```

Any request to exceed 2 remote days per week requires written approval from the employee's manager and HR Business Partner. Remote work days must be logged in the HR portal by Friday of each week.

Recovery actions:

```
read_policy(remote_policy_BAD) -> {'error': 'PolicyNotFoundError', 'message': 'Policy document not found: remote_policy_BAD'}
policy_search('How many remote days are allowed?', k=3) -> [{'doc_id': 'remote_work_v3', 'score': 0.2512838840484619, 'date': '2024-01-15', 'preview': 'Remote Work Policy'}]
read_policy(remote_work_v3) -> Remote Work Policy (v3): Employees in eligible roles may work remotely up to 2 days per week. Any request to exceed 2 remote days per week requires written approval from the employee's manager and HR Business Partner. Remote work days must be logged in the HR portal by Friday of each week.
read_policy(laptop_equipment_v1) -> Laptop & Equipment Policy (v1): The company issues one standard laptop per employee, refreshed every 3 years.
```

What I noticed: Keeping the failed read in the trace is useful. It proves that recovery occurred rather than making the final correct answer look like a normal first-attempt retrieval.

Task 1.5 — Evaluation Version 1

I created nine canned employee questions covering every current policy family. Each run records latency, agent steps, graph steps, document-ID presence, critic validity, pass/fail, memory status, stop reason, answer text, and the full execution trace.

Colab Output – Evaluation Summary

```
[ ] from run_evaluation import evaluate
    summary=evaluate()
    print("SUMMARY",summary)
```

Output

```
Q1: latency=1.085 ms | steps=3 | has_docid=True | pass=True
Q2: latency=1.100 ms | steps=3 | has_docid=True | pass=True
Q3: latency=0.892 ms | steps=3 | has_docid=True | pass=True
Q4: latency=0.948 ms | steps=3 | has_docid=True | pass=True
Q5: latency=0.850 ms | steps=2 | has_docid=True | pass=True
Q6: latency=0.927 ms | steps=3 | has_docid=True | pass=True
Q7: latency=0.891 ms | steps=3 | has_docid=True | pass=True
Q8: latency=0.861 ms | steps=3 | has_docid=True | pass=True
Q9: latency=0.886 ms | steps=3 | has_docid=True | pass=True
SUMMARY {'questions': 9, 'passed': 9, 'pass_rate': 1.0, 'average_latency_ms': 0.9377777777777777}
```

Figure 5. Colab evaluation console showing all nine questions passing.

ID	Expected document	Steps	Critic	Result
Q1	remote_work_v3	3	Valid	Pass
Q2	leave_policy_v2	3	Valid	Pass
Q3	travel_expense_v4	3	Valid	Pass
Q4	security_mfa_v1	3	Valid	Pass
Q5	password_policy_v2	2	Valid	Pass
Q6	laptop_equipment_v1	3	Valid	Pass
Q7	data_retention_v2	3	Valid	Pass
Q8	expense_approval_v1	3	Valid	Pass
Q9	vpn_access_v2	3	Valid	Pass

Recorded result: 9 of 9 questions passed. The mean local runtime was 0.938 ms in this execution environment. These timings describe a tiny local corpus and should not be treated as production LLM latency.

#SCREENSHOT & OUTPUT Task 1.5 :-

Task 1.5 — Evaluation Version 1

I evaluated nine canned questions across the whole corpus. Each row records latency, tool steps, graph steps, document-ID presence, critic status, memory status, and pass/fail. Full traces are saved as JSONL.

```
from run_evaluation import evaluate
summary = evaluate()
print("\nSUMMARY")
print(json.dumps(summary, indent=2))
```

```
Q1: latency=1.156 ms | steps=3 | has_docid=True | pass=True
Q2: latency=1.402 ms | steps=3 | has_docid=True | pass=True
Q3: latency=1.169 ms | steps=3 | has_docid=True | pass=True
Q4: latency=1.122 ms | steps=3 | has_docid=True | pass=True
Q5: latency=1.060 ms | steps=2 | has_docid=True | pass=True
Q6: latency=2.820 ms | steps=3 | has_docid=True | pass=True
Q7: latency=1.130 ms | steps=3 | has_docid=True | pass=True
Q8: latency=1.166 ms | steps=3 | has_docid=True | pass=True
Q9: latency=1.078 ms | steps=3 | has_docid=True | pass=True
```

```
SUMMARY
{
  "questions": 9,
  "passed": 9,
  "pass_rate": 1.0,
  "average_latency_ms": 1.3447777777777778
}
```

```
import pandas as pd
results_df = pd.read_csv("logs/evaluation_results.csv")
results_df[["id", "expected_doc_id", "latency_ms", "agent_steps", "has_docid", "critic_valid", "pass"]]
```

...	id	expected_doc_id	latency_ms	agent_steps	has_docid	critic_valid	pass
0	Q1	remote_work_v3	1.156	3	True	True	True
1	Q2	leave_policy_v2	1.402	3	True	True	True
2	Q3	travel_expense_v4	1.169	3	True	True	True
3	Q4	security_mfa_v1	1.122	3	True	True	True
4	Q5	password_policy_v2	1.060	2	True	True	True
5	Q6	laptop_equipment_v1	2.820	3	True	True	True
6	Q7	data_retention_v2	1.130	3	True	True	True
7	Q8	expense_approval_v1	1.166	3	True	True	True
8	Q9	vpn_access_v2	1.078	3	True	True	True

Task 2.1 — Verified Memory

The memory uses a second FAISS inner-product index with deterministic hashing vectors. A record is stored only when the critic returns valid: true and the answer has at least one document citation.

A warm memory match is the cheapest sufficient candidate, not final evidence. The agent uses its document ID as a hint and calls read_policy again before making a claim. This protects the answer from treating stored text as a replacement for source verification.

Colab Output – Verified Memory

```
[ ] critic=GroundingCritic(tools);memory=VerifiedFactMemory();q="How often must corporate passwords be rotated?"
verified=agent.answer(q);verdict=critic.evaluate(verified)
print("Critic:",verdict,"Written:",memory.write(q,verified,verdict["valid"]))
print("Warm hint:",memory.read(q))
```

Output

```
Critic: {'valid': True, 'issues': []} Written: True
Warm hint: {'question': 'How often must corporate passwords be rotated?', 'answer': 'Multi-Factor Authentication (MFA) is mandatory for all VPN connections into the corporate network, for all SaaS applications holding customer data, and for privileged administrator accounts. [security_mfa_v1] Supporting text: "Multi-Factor Authentication (MFA) is mandatory for all VPN connections into the corporate network, for all SaaS applications holding..." [security_mfa_v1] Corporate account passwords must be at least 14 characters long and include a mix of uppercase, lowercase, numbers, and symbols. Passwords must be rotated every 180 days and must not match any of the previous 10 passwords used. [password_policy_v2] Supporting text: "Corporate account passwords must be at least 14 characters long and include a mix of uppercase, lowercase, numbers,..." [password_policy_v2]', 'doc_ids': ['security_mfa_v1', 'password_policy_v2'], 'score': 0.9999999403953552}
```

Figure 6. Colab output showing critic approval, a successful verified write, and a warm memory hint.

#SCREENSHOT & OUTPUT Task 2.1 :-

Task 2.1 — Verified FAISS Memory

The memory writes only when the critic has accepted the answer and at least one document ID is present. A warm memory result is only a candidate hint; the agent still reads the source document before answering.

```
from policy_assistant import GroundingCritic, VerifiedFactMemory
critic = GroundingCritic(tools)
memory = VerifiedFactMemory()

valid = agent.answer("How often must corporate passwords be rotated?")
verdict = critic.evaluate(valid)
print("Valid answer critic:", verdict)
print("Written:", memory.write("How often must corporate passwords be rotated?", valid, verdict["va
print("Warm read:", memory.read("How often must corporate passwords be rotated?"))

invalid = type(valid)("Unsupported answer", [], [])
print("Invalid write accepted:", memory.write("unsupported", invalid, False))
```

```
Valid answer critic: {'valid': True, 'issues': []}
Written: True
Warm read: {'question': 'How often must corporate passwords be rotated?', 'answer': 'Multi-Factor Au
Invalid write accepted: False
```

Task 2.2 — Graph Orchestration

I implemented the route directly so each transition remains inspectable. The graph counts node visits, permits one critic retry, and stops safely when its budget is exhausted.

Colab Output — Graph and Critic

```
[ ] graph=PolicyGraph(agent,critic,VerifiedFactMemory(),8,1)
g=graph.run("How is VPN access approved, and when does an inactive session time out?")
print(g.answer)
print("Route:", " -> ".join(x["node"] for x in g.trace if x["node"]!="agent_trace"))
print("Critic:",g.critic)
```

Output

VPN sessions automatically time out after 30 minutes of inactivity and require MFA re-authentication as described in the Information Security Policy. Access requests must be approved by the employee's manager and provisioned by IT within 2 business days. [vpn_access_v2] Supporting text: "VPN sessions automatically time out after 30 minutes of inactivity and require MFA re-authentication as described in the..." [vpn_access_v2] Multi-Factor Authentication (MFA) is mandatory for all VPN connections into the corporate network, for all SaaS applications holding customer data, and for privileged administrator accounts. [security_mfa_v1] Supporting text: "Multi-Factor Authentication (MFA) is mandatory for all VPN connections into the corporate network, for all SaaS applications holding..." [security_mfa_v1]
Route: plan -> memory_read -> agent_answer -> critic -> memory_write -> stop
Critic: {'valid': True, 'issues': []}

Figure 7. Colab output showing the VPN answer, graph route, and valid critic result.

#SCREENSHOT & OUTPUT Task 2.2 :-

Task 2.2 — Graph Orchestration

The graph route is `plan → memory_read → agent_answer → critic → memory_write → stop`. It permits one critic retry and stops when the graph budget is exhausted.

```
import sys
import os
import importlib

# Force current directory into system path to resolve module location
sys.path.insert(0, os.path.abspath("/content"))
importlib.invalidate_caches()

from policy_assistant import PolicyGraph

graph = PolicyGraph(agent, critic, VerifiedFactMemory(), max_graph_steps=8, max_retries=1)
graph_result = graph.run("How is VPN access approved, and when does an inactive session time out?")
print(graph_result.answer)
print("\nGraph route:", " -> ".join(event["node"] for event in graph_result.trace if event["node"] != "agent_trace"))
print("Critic:", graph_result.critic)
print("Stop reason:", graph_result.stopped_reason)
```

VPN sessions automatically time out after 30 minutes of inactivity and require MFA re-authentication as described in the Information Security Policy. Access requests must be approved by the employee's manager and provisioned by IT within 2 business days.

Graph route: plan -> memory_read -> agent_answer -> critic -> memory_write -> stop
Critic: {'valid': True, 'issues': []}
Stop reason: completed

Cold and warm routes

I run the same question twice. The second run should report `memory_hit=True`, while still validating the hinted document through `read_policy`.

```
import time
from policy assistant import PolicyTools, PolicyAgent, GroundingCritic, VerifiedFactMemory, PolicyGraph

# Initialize dependencies to prevent NameError
tools = PolicyTools("policies.jsonl")
agent = PolicyAgent(tools, max_steps=4)
critic = GroundingCritic(tools)
memory = VerifiedFactMemory()
graph = PolicyGraph(agent, critic, memory, max_graph_steps=8, max_retries=1)

cold_start = time.perf_counter(); cold = graph.run("How often must corporate passwords be rotated?"); cold_ms = (time.perf_counter()-cold_start)*1000
warm_start = time.perf_counter(); warm = graph.run("How often must corporate passwords be rotated?"); warm_ms = (time.perf_counter()-warm_start)*1000
print({"cold_ms": round(cold_ms,3), "cold_memory_hit": cold.memory_hit,
      "warm_ms": round(warm_ms,3), "warm_memory_hit": warm.memory_hit,
      "warm_agent_steps": warm.agent_steps})

... {'cold_ms': 1.446, 'cold_memory_hit': False, 'warm_ms': 0.826, 'warm_memory_hit': True, 'warm_agent_steps': 1}
```

#Budget Stop

For the acceptance test, I reduced the graph budget to two nodes. Execution stopped after planning and memory read instead of continuing into answer generation.

Colab Output – Budget Stop

```
[ ] tiny=PolicyGraph(agent,critic,VerifiedFactMemory(),2,1).run("What is annual leave?")
    print("Budget result:",tiny.stopped_reason,tiny.critic)
```

Output

```
Budget result: budget_exhausted {'valid': False, 'issues': ['graph_budget_exhausted']}
```

Figure 8. Colab output showing the explicit budget-exhausted result.

SCREENSHOT & OUTPUT (Budget Stop):-

Budget stop

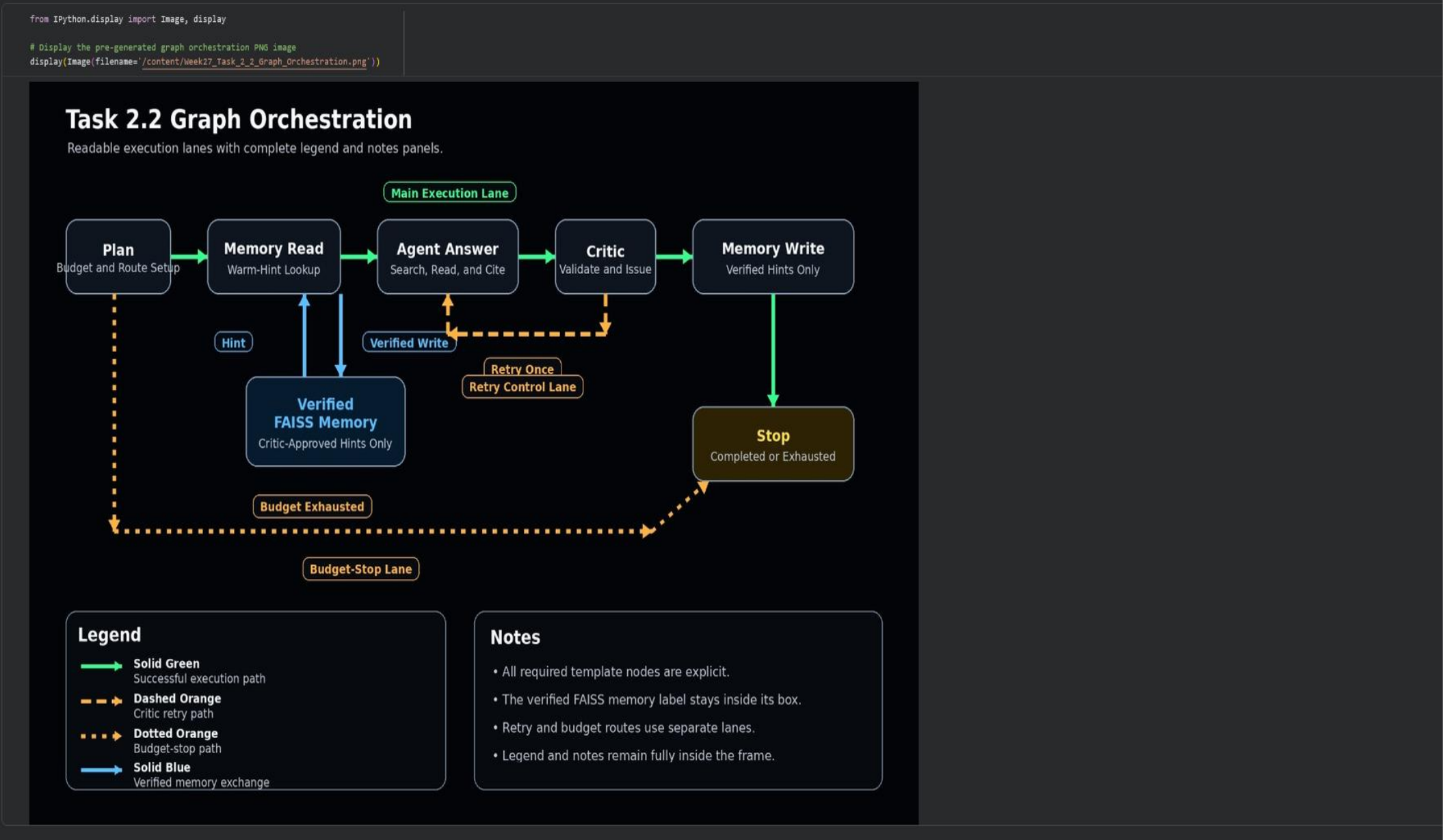
A deliberately tiny graph budget demonstrates that the route stops predictably instead of entering an uncontrolled loop.

```
small_budget_graph = PolicyGraph(agent, critic, VerifiedFactMemory(), max_graph_steps=2)
budget_result = small_budget_graph.run("What is the annual leave policy?")
print(budget_result.stopped_reason)
print(budget_result.critic)
print(budget_result.trace)
```

budget_exhausted

```
{'valid': False, 'issues': ['graph_budget_exhausted']}
```

```
[{'node': 'plan', 'details': {'route': 'memory_read -> agent_answer -> critic'}}, {'node': 'memory_read', 'details': {'hit': False, 'hint': None}}, {'node': 'stop', 'details': {'reason': 'budget_exhausted'}}]
```



Task 2.3 — Critic

The critic returns the required structure `{valid: bool, issues: [] }`. It checks that a bracketed document ID exists, a quotation is present, every cited ID exists in the corpus, and at least one stored quotation occurs in the cited source.

Check	Failure Issue	Purpose
Document token	missing_doc_id	Prevents uncited policy claims.
Quotation	missing_quote	Requires visible source evidence.
Known source	unknown_doc_id	Rejects fabricated IDs.
Quote support	quote_not_supported	Checks that quoted text occurs in the source.

This critic is deliberately narrow. It validates the project acceptance contract, but it is not a complete natural-language entailment judge. A production extension should check each claim separately against the exact cited sentence.

#SCREENSHOT & OUTPUT Task 2.3 :-

Task 2.3 — Critic

The critic returns the required JSON shape: `{"valid": bool, "issues": []}`. It checks document-token presence, quotation presence, document existence, and whether a stored quotation occurs in the cited source.

```
good_verdict = critic.evaluate(agent.answer("What is the India meal allowance?"))
bad_result = type(valid)("Employees may claim anything they want.", [], [])
print("GOOD:", json.dumps(good_verdict, indent=2))
print("BAD:", json.dumps(critic.evaluate(bad_result), indent=2))
```

```
GOOD: {
  "valid": true,
  "issues": []
}
BAD: {
  "valid": false,
  "issues": [
    "missing_doc_id",
    "missing_quote"
  ]
}
```

Task 2.4 – Tests and Documentation

The final PyTest suite covers retrieval ranking, the typed missing-ID error, acceptance rules, bad-ID recovery, verified memory, and graph-budget stopping. The executable notebook also retains all code and outputs in task order.

Colab Output – PyTest

```
[ ] import pytest
    exit_code=pytest.main(["-q","tests"])
    print("pytest exit code:",exit_code)
    assert exit_code==0
```

Output

```

[][32m.[][0m
[][32m.[][0m
[][32m.[][0m
[][32m.[][0m
[][32m.[][0m
[][32m.[][0m
[][32m.[][0m
[][32m

[100%][][0m

[][32m[][32m[][1m6 passed[][0m[][32m in 0.03s[][0m[][0m
pytest exit code: 0

```

Figure 9. Colab output confirming all six automated tests passed.

Test area	What is verified
Search	The current leave policy ranks first for carry-over.
Typed error	A missing record raises PolicyNotFoundError.
Acceptance	A grounded answer contains the expected ID and quotation.
Recovery	A bad ID falls back to search and reaches remote_work_v3.
Memory	The repeated verified question produces a warm hit.
Budget	A two-node graph stops with budget_exhausted.

Submission Files

The project package includes the executed Colab notebook, supplied JSONL data, modular Python source, requirements, evaluation runner, CSV/JSONL/JSON logs, PyTests, README, and this PDF report.

Results and Discussion

Retrieval Behaviour

Distinctive vocabulary makes the ten-policy corpus well suited to transparent lexical retrieval. The leave case is the main ambiguity: both versions contain annual-leave language, so superseded-version handling is necessary in addition to vector ranking.

Grounding Behaviour

The extractive answer generator selects source sentences instead of asking a remote model to paraphrase them. This reduces conversational variety, but it makes repeated runs stable and gives the critic a concrete quotation to verify.

Recovery and Memory

The typed error distinguishes an invalid document ID from an unsupported question. Verified memory shortens a repeated route by suggesting a document ID, while source re-reading preserves grounding. These two controls solve different problems: recovery handles failed access, while memory improves repeated access without bypassing evidence.

What I Noticed

The project made it clear to me that agent reliability comes from several small controls working together. Retrieval alone cannot guarantee a supported answer. Exact citations, quotations, a critic, gated memory writes, retry limits, and budget stops each remove a different failure mode.

- Direct employee wording works well because the corpus contains distinctive terms such as MFA, VPN, carry-over, and meal allowance.
- The leave-policy case is important because retrieval sees both versions; explicit superseded-version handling prevents an old policy from becoming the answer.
- Exact document tokens and short quotations make the grounding contract easy to inspect.
- Verified memory reduces the warm route, but source re-reading keeps the answer grounded.
- Explicit budgets make failures safe and easy to test.
- Actual latency depends on the Colab runtime, so the CSV produced by this notebook is the correct evidence for the submitted run.

#Limitations

TF-IDF is based mainly on vocabulary overlap and does not provide deep semantic understanding.

The memory is in-process and resets with the runtime. The critic verifies IDs and quotations but does not prove every logical claim. The extractive answer style is dependable for this assignment but less flexible than a tightly controlled LLM.

- F-IDF depends mainly on vocabulary overlap and does not provide deep semantic understanding.
- Memory is in-process and resets when the runtime restarts
- The critic verifies citations and quotations, but it is not a complete natural-language entailment "MS-SQL".
- Extractive answers are reproducible but less conversational than a constrained LLM.

#Possible Extension

If I extended this work, I would compare the current TF-IDF retriever with sentence-transformer embeddings, add "MS-SQL" based checkpointing, and introduce claim-by-claim critic checks to validate each claim against its cited source sentence.

I would retain the existing typed errors, trace logging, verification gates, retry limits, and budget tests, as these controls remain valuable for reliability and reproducibility regardless of the underlying retrieval or generation model.

Final Project Summary

Week 27 Graded Mini Project — Policy Assistant Agent

In this project, I built a complete grounded Policy Assistant around the supplied local policy corpus. I implemented normalized TF-IDF vectors in FAISS, a search tool, a full-document reader with a typed error, an optional safe calculator, deterministic ReAct-style traces, bad-ID recovery, verified-only memory, budgeted graph orchestration, a minimal JSON critic, structured evaluation logs, and automated tests.

The recorded run answered all nine evaluation questions with the expected document IDs and valid critic outcomes. The bad-ID scenario recovered through search, the memory scenario produced a verified warm hint, the low-budget scenario stopped explicitly, and all six PyTests passed.

The strongest learning for me was that the final answer is only one part of a dependable agent. The surrounding route—search, source reading, exact citations, verification, memory write conditions, retries, and stop rules—determines whether that answer is auditable and safe to reuse.

In this project, I built a complete local Policy Assistant around the supplied dataset. I implemented FAISS retrieval, policy tools, deterministic ReAct-style traces, bad-ID recovery, verified-only memory, a budgeted graph with one critic retry, structured evaluation logs, and PyTests.

The main learning for me was that reliable agent behaviour depends on the route around the answer as much as the answer itself. Search, source reading, exact citations, verification, memory write conditions, retry limits, and stop rules each remove a different failure mode. The final system is intentionally lightweight, but its behaviour is visible, reproducible, and easy to test.

Submission requirement	Included
Local JSONL policy corpus	Yes — 10 supplied records
FAISS retrieval and tools	Yes
Agent and bad-ID recovery	Yes
Verified memory and graph	Yes
Critic and budgets	Yes
Evaluation logs and screenshots	Yes
PyTests and README	Yes
Executed Colab notebook	Yes
Project PDF report	Yes

End of Week 27 Graded Mini Project